



docker ps Command

Introduction

The **docker ps** command is used to **list and monitor** running Docker containers. It helps users:

- Check running containers
- Monitor resource usage
- Identify container IDs for management

This guide explains **how to use docker ps effectively** with practical examples.

Section 1: Learn

What is **docker ps**?

docker ps is a **Docker CLI command** that shows a list of **running containers** with useful details like:

Column	Description
CONTAINER ID	Unique ID for the container
IMAGE	Docker image used to create the container
COMMAND	Command executed inside the container
CREATED	Time since the container was created
STATUS	Running or stopped state



Column	Description
PORTS	Port mappings between host and container
NAMES	Name assigned to the container

Why is **docker ps** Important?

Use Case	Benefit
Monitor Running Containers	View which containers are active
Get Container IDs	Needed for stopping/removing containers
Check Logs	Debug container issues
Inspect Resource Usage	Analyze CPU and memory usage

Interesting Fact

By default, **docker ps** only shows running containers. To see all containers, including stopped ones, use **docker ps -a**.

Section 2: Practice

1. Listing Running Containers

```
docker ps
```

Example Output:

```
CONTAINER ID  IMAGE  COMMAND  STATUS  PORTS
NAMES
```



```
f1a2b3c4d5e6  nginx  "nginx -g ..." Up 5 mins  0.0.0.0:8080->80/tcp
web_server
```

This shows:

- A running Nginx container
 - Mapped port 8080 on the host to 80 in the container
 - The container's name is `web_server`
-

2. Listing All Containers (Including Stopped Ones)

```
docker ps -a
```

This displays **all containers**, including ones that have stopped.

Why is this useful?

- Helps **find old containers** for cleanup.
 - Provides history of **stopped or failed** containers.
-

3. Displaying Only Container IDs

```
docker ps -q
```

This returns only container IDs, useful for **bulk operations**:

```
docker stop $(docker ps -q)
```

Stops all running containers.

4. Filtering Containers

To list only containers from a specific image:



```
docker ps --filter "ancestor=nginx"
```

To show only containers created in the last **1 hour**:

```
docker ps --filter "since=1h"
```

5. Viewing Containers with Custom Formatting

```
docker ps --format "table {{.ID}}    {{.Image}}  {{.Status}}"
```

Output:

```
CONTAINER ID  IMAGE  STATUS
f1a2b3c4d5e6  nginx  Up 5 minutes
```

Why is this useful?

- Customizes output for **better readability**.

Section 3: Know More

Frequently Asked Questions

1. What is the difference between **docker ps** and **docker ps -a**?
 - **docker ps** shows only running containers.
 - **docker ps -a** shows all containers (running + stopped).
2. How do I stop all running containers?

```
docker stop $(docker ps -q)
```

3. How do I find a container's status?

```
docker ps --filter "name=my_container"
```



4. Can I view only exited containers?

```
docker ps -a --filter "status=exited"
```

5. How do I remove all stopped containers?

```
docker container prune
```

Conclusion

The **docker ps** command is essential for monitoring, managing, and troubleshooting containers.

Start by **listing running containers**, **filtering output**, and **managing stopped containers**, and soon you'll be **efficiently handling Docker workloads!**